Invoice Data Extraction Backend – Technical Documentation

August 28, 2025

Contents

1 Overview **2**

2 Architecture & Flow **2**

3 Data Model Examples **3**

4 Core Modules & Functions **4**

5 API Reference **4**

5.1 POST /upload 4
5.2 GET /files 4
5.3 POST /process/{file_id} 4
5.4 GET /extraction/{file_id} 5
5.5 POST /structure/{file_id} 5
5.6 PUT /update/{file_id} 5
5.7 DELETE /files/{file_id} 5

6 Error Handling & Status Codes **6**

1 Overview

This backend provides an end-to-end pipeline for invoice and delivery note ingestion, text/markdown extraction, and structured JSON field extraction using a local LLM. Uploaded files are stored in MongoDB GridFS; extraction artifacts and structured data are persisted in dedicated collections.

Key Features

- Upload PDFs/Images → stored in GridFS with metadata.
- Smart extraction:
  - Fast path via embedded PDF text layer (no OCR).
  - OCR path via Marker pipeline with optional image serialization.
- LLM post-processing (Ollama `mistral`) to produce structured invoice JSON.
- Idempotent processing (skip if already processed unless `force_ocr=true`).
- Update route to apply human corrections.

Primary Technologies

FastAPI (async), Motor/Async GridFS, PyPDF, Marker, Ollama, Pillow, MongoDB.

2 Architecture & Flow

1. **POST /upload**: store file in GridFS; `processed=false`.
2. **POST /process/{file_id}**:
   - If PDF has readable text and `force_ocr=false`: extract via PyPDF (fast path).
   - Else: run Marker OCR; extract Markdown and base64 images.

   Save result to `extractions`; mark file processed.
3. **POST /structure/{file_id}**: send extracted text/markdown to LLM; store JSON in `structured_data`.
4. **PUT /update/{file_id}**: store user edits back to `structured_data`.
5. **GET /extraction/{file_id}**: fetch extraction artifact for a file.
6. **GET /files** & **DELETE /files/{file_id}**: list & delete uploads.

Collections

- GridFS: `fs.files`, `fs.chunks`
- `extractions`: {`file_id`, `original_filename`, `extraction_data`, `created_at`}
- `structured_data`: {`file_id`, `structured_json`, `created_at`}

3 Data Model Examples

GridFS file (`fs.files`)


```
"_id": ObjectId("656f2a1b8e0f2c8d37b12345"),
"filename": "invoice.pdf",
"length": 254318,
"uploadDate": ISODate("2025-08-28T19:00:00Z"),
"metadata":
"upload_date": ISODate("2025-08-28T19:00:00Z"),
"content_type": "application/pdf",
"original_filename": "invoice.pdf",
"processed": true,
"processed_at": ISODate("2025-08-28T19:03:12Z")
```


Extraction document (`extractions`)


```
"_id": ObjectId("65702a9c2e0f2c8d37c67890"),
"file_id": ObjectId("656f2a1b8e0f2c8d37b12345"),
"original_filename": "invoice.pdf",
"extraction_data":
"file_name": "invoice",
"original_path": "invoice.pdf",
"content": "Invoice #123: 2025-08-28...",
"images": [],
"images_count": 0,
"extraction_timestamp": 1693241234.123,
"processing_time": 1.42,
"extraction_mode": "pdf_text_layer"
,
"created_at": ISODate("2025-08-28T19:03:12Z")
```


Structured data document (`structured_data`)


```
"_id": ObjectId("65702bca3d0f2c8d37a5aa11"),
"file_id": ObjectId("656f2a1b8e0f2c8d37b12345"),
"structured_json":
"document_type": "Facture",
"currency": "MAD",
"payment_method": "Virement Bancaire",
"invoice_number": "F-2025-0087",
"invoice_date": "28.08.2025",
"due_date": "",
"total_amount": "12500.00",
"tax_amount": "2250.00",
"line_items": [
"name":"Pompe industrielle","quantity":"2","unit_price":"5000.00",
"packaging":"","unit":"pcs","total_ht":"10000.00"
]
,
"created_at": ISODate("2025-08-28T19:04:45Z")
```

4 Core Modules & Functions

Readable-PDF Fast Path

__pdf_is_readable(pdf_bytes, min_chars=250, sample_pages=1) → **bool**: sample pages with PyPDF; if char count ≥ threshold, skip OCR.
__pdf_extract_text(pdf_bytes) → **str**: extract text from all pages and join with `\n\n`.

Marker Models & Rendering

get_marker_models() (async): lazy-load Marker artifacts once (global cache).
extract_markdown_content(rendered): normalize renderer output to markdown.
image_to_base64(img): serialize PIL images to base64 PNG (or pass paths).

Unified Extraction

extract_content_from_file(file_content, original_filename, force_ocr=False) (async): decision tree → readable PDF (PyPDF) or Marker OCR; returns unified payload with `content`, `images`, `images_count`, `extraction_mode`, `processing_time`.

5 API Reference

5.1 POST /upload

Description: Store a file in GridFS with metadata.
Request (multipart/form-data): `file` (PDF, PNG, JPEG, ...)
Response 200:

```
"message":"File uploaded successfully","file_id":"656f2a1b8e0f2c8d37b12345"
```

5.2 GET /files

Description: List up to 100 uploaded files (GridFS metadata).
Response 200:

```
"files":
    ["id":"...",
    "filename":"...",
    "upload_date":"...",
    "content_type":"...",
    "length":12345,"processed":false]
```

5.3 POST /process/{file_id}

Description: Extract text/markdown and images; upsert into `extractions`; mark file processed.
Query params: `force_ocr` (bool, default `false`)
Response 200:

```
    "message": "File processed successfully",
    "extraction_id": "65702a9c2e0f2c8d37c67890",
    "extraction":
    "file_name": "invoice",
    "original_path": "invoice.pdf",
    "content": "...",
    "images": [],
    "images_count": 0,
    "extraction_timestamp": 1693241234.123,
    "processing_time": 1.42,
    "extraction_mode": "pdf_text_layer"
```

5.4 GET /extraction/{file_id}

Description: Fetch extraction document for a given file.

Response 200:

```
"extraction":
    "_id":"...",
    "file_id":"...",
    "original_filename":"...",
    "extraction_data":  ... ,
    "created_at":"..."
```

5.5 POST /structure/{file_id}

Description: Use LLM (Ollama `mistral`) to convert extracted text to structured invoice JSON; insert into `structured_data`.

Response 200:

```
"message":"Structured data extracted and saved.", "data":  "document_type":"Facture",
↪  ...
```

5.6 PUT /update/{file_id}

Description: Persist user-edited structured JSON for a file.

Request body:

```
"document_type":"Facture", "currency":"MAD", ...
```

Response 200:

```
"message": "Données mises à jour avec succès"
```

5.7 DELETE /files/{file_id}

Description: Delete file from GridFS (and clean related docs).

Response: 204 No Content

Recommended cleanup:

```
await db.extractions.delete_many("file_id": oid)
await db.structured_data.delete_many("file_id": oid)
```

6 Error Handling & Status Codes

- 400: invalid ObjectId, missing file.
- 404: file not found; extraction/structured data missing.
- 500: DB/IO errors, OCR/LLM failures, invalid JSON returned by LLM.